CODE QUEST / JAVASCRIPT / 4e

Le corrigé complet

Construire son jeu, comprendre son code.

40 missions · 12 bilans QCM · 26 questions



```
const playerName = "Nova";

player.x = player.x + speed;
```

Toutes les corrections dans l'ordre de l'atelier : code à ajouter ou à modifier, essais pratiques, critères de réussite et réponses expliquées aux QCM.

Inclus : le fichier game.js complet et un repère sur les erreurs les plus fréquentes.

Document de correction pour l'enseignant. Les choix personnels des élèves restent possibles lorsque la mission les autorise.

Se repérer dans le corrigé

Les numéros M01 à M40 et Q01 à Q12 sont ceux de l'atelier. Les bilans sont intercalés au même endroit que dans le parcours.

Utiliser le corrigé . . . 3

01. Le héros . . . 4

02. Les premiers pas . . . 7

03. Les commandes . . . 10

04. Les sauts . . . 14

05. Le score . . . 17

06. Le trophée . . . 20

07. Le drapeau . . . 23

08. Ta version du jeu . . . 26

Réponses QCM : grille rapide . . . 29

Le fichier game.js complet . . . 30

Cas de test et erreurs fréquentes . . . 32

Deux types de mission, une même progression

20 missions de code. L'élève ajoute une donnée, complète une fonction ou modifie une règle dans le même fichier `game.js`.

20 défis pratiques. L'élève compare des essais, joue un trajet ou éprouve une règle. Le code peut être juste avant que le défi soit terminé.

12 bilans QCM. Les 26 questions sont reproduites avec les trois propositions et une explication pour chaque réponse.

Corriger sans imposer la copie

Les blocs de code montrent des solutions possibles. Le résultat et la compréhension de la règle comptent autant que son écriture.

Garder un seul fichier

Toutes les missions construisent le même `game.js`. Lorsqu'une mission demande de modifier une ligne ou une fonction, on corrige celle qui existe déjà. On conserve le code précédent.

Avec `const`, l'élève change la valeur dans le texte du fichier, puis vérifie pour relancer le jeu. Il ne réaffecte pas cette constante pendant une partie.

En particulier, `updateX` évolue en trois temps : avancer, avancer avec la flèche droite, puis avancer dans les deux sens. À la fin, une seule version de cette fonction reste dans le fichier.

Accepter les variantes qui fonctionnent

Les espaces, les retours à la ligne et les guillemets simples ou doubles peuvent varier. Des écritures équivalentes, comme `player.x += speed;` ou `2 * count`, peuvent produire la même règle. Les identifiants que le jeu lit doivent rester corrects.

Le vérificateur essaie les fonctions avec plusieurs valeurs. Une réponse qui réussit un seul exemple en renvoyant un nombre fixe ne démontre pas la règle.

Distinguer code valide et défi accompli

Le bouton Vérifier contrôle le code puis lance le jeu si tout est valide. Un défi pratique attend ensuite l'action demandée : saut, récolte, aller-retour, victoire ou comparaison. Les essais de comparaison sont conservés entre les vérifications.

Toute modification du code impose une nouvelle vérification pour jouer. Pendant un QCM, le fichier reste visible en lecture seule. Une réponse fausse peut être reprise ; toutes les réponses du bilan doivent être comprises et validées.

Pour aider l'élève	Question courte à lui poser
Avant une modification	« Qu'est-ce que tu penses voir changer ? »
Après un essai	« Qu'est-ce qui a changé dans le jeu ? »
Pour vérifier la compréhension	« Quelle ligne explique ce résultat ? »
Devant une erreur	« Quel cas ne fonctionne pas encore ? »

CHAPITRE 01 / M01-M05 / Q01 · Q02

Le héros

Nommer une donnée, distinguer texte et nombre, puis placer le héros sur l'axe X.

M01 / CODE

Nomme ton héros

Consigne. Choisis un nom court pour ton héros. Écris ta ligne avec `playerName` dans le fichier vide.

Correction. Ajoute cette ligne dans le fichier vide. Le nom entre guillemets est libre.

```
const playerName = "Ninja";
```

À comprendre. `const` déclare une donnée nommée `playerName`. Le jeu lit sa valeur pour afficher le nom du héros.

Réponses acceptées. Un texte de 2 à 16 caractères après suppression des espaces aux extrémités. Le nom « player » est refusé.

Test et réponse au défi. Changer le texte puis vérifier change le nom affiché. Les espaces autour de `=` et le type de guillemets peuvent varier.

Erreur fréquente. Ne remplace pas l'identifiant `playerName` et ne supprime pas les guillemets autour du texte.

M02 / DÉFI PRATIQUE

Deux noms, un seul héros

Consigne. Choisis un nouveau nom et vérifie. Change encore le nom sur la même ligne, puis vérifie.

Un essai possible. Sur la même ligne : `const playerName = "Nova";`, puis `const playerName = "Luna";`.

À faire dans le jeu. Cliquer sur Vérifier après chaque nom. Observer le changement du nom affiché.

Validation attendue. Deux noms différents après suppression des espaces aux extrémités. Ajouter seulement des espaces autour du même nom ne compte pas.

À expliquer. Une information peut être modifiée sans créer une deuxième déclaration. Le résultat dépend de la valeur, pas de l'ajout d'une ligne.

Q01 / BILAN QCM

Lire une ligne

Q01 / Question 1. Que contient playerName ?

```
const playerName = "Nova";
```

A **Le texte Nova RÉPONSE CORRECTE**
Les guillemets entourent le texte choisi pour le héros.

B **Le nombre de vies**
Cette ligne contient un nom entre guillemets, pas un nombre de vies.

C **Une touche du clavier**
Aucune touche n'est testée ici : on donne un nom au héros.

Q01 / Question 2. Pour changer le nom du héros, quelle partie modifies-tu ?

```
const playerName = "Nova";
```

A **Le mot const**
const sert à créer la donnée. Le nom affiché est le texte entre guillemets.

B **Le texte entre guillemets RÉPONSE CORRECTE**
Le jeu continue de lire playerName, avec le nouveau texte.

C **Le point-virgule**
Le point-virgule termine la ligne ; il ne choisit pas le nom.

M03 / CODE

Choisis son départ

Consigne. Ajoute une ligne avec `startX`. Choisis un départ entre `24` et `70`.

Correction. Ajoute une nouvelle ligne, en gardant `playerName`.

```
const startX = 43;
```

À comprendre. `startX` règle la position horizontale de départ. Plus X est grand, plus le départ est à droite.

Réponses acceptées. Un nombre de 24 à 70, bornes incluses, sans guillemets. 43 est seulement un exemple.

Test et réponse au défi. Avec 30 puis 60, le départ se décale vers la droite. Cela ne modifie pas la hauteur.

Erreur fréquente. `"43"` est du texte ; `43` est un nombre.

M04 / DÉFI PRATIQUE

Deux points de départ

Consigne. Essaie un départ près de `30` et vérifie. Sur la même ligne, essaie près de `60` et vérifie.

Un essai possible. Tester `const startX = 30;`, puis modifier la même ligne en `const startX = 60;`.

À faire dans le jeu. Vérifier chaque valeur et comparer les deux positions de départ.

Validation attendue. Deux départs valides séparés d'au moins 15 pixels. Chaque valeur doit rester entre 24 et 70.

À expliquer. 60 place le héros plus à droite que 30. Seule la position initiale change.

M05 / DÉFI PRATIQUE

Gare ton héros

Consigne. Modifie `startX` pour partir entre `50` et `60`. Vérifie et regarde la position X sous le jeu.

Un essai possible. Modifier la ligne existante, par exemple : `const startX = 55;`.

À faire dans le jeu. Vérifier puis lire la position X affichée sous le jeu.

Validation attendue. Le départ doit être compris entre 50 et 60, bornes incluses.

À expliquer. Le nombre correspond à une position horizontale. Plusieurs réponses sont correctes dans la zone demandée.

Q02 / BILAN QCM

Comprendre la position

Q02 / Question 1. On passe startX de 30 à 60. Où commence le héros ?

```
const startX = 60;
```

A **Plus à gauche**
Sur l'axe X, un nombre plus grand place le héros plus à droite.

B **Plus à droite RÉPONSE CORRECTE**
60 est plus grand que 30 : le départ se décale vers la droite.

C **Plus haut**
startX règle l'horizontale. La hauteur correspond à l'axe Y.

Q02 / Question 2. Quelle valeur est bien un nombre ?

A **"50"**
Avec les guillemets, "50" est du texte.

B **"cinquante"**
Un mot entre guillemets est du texte.

C **50 RÉPONSE CORRECTE**
Sans guillemets, 50 est un nombre utilisable pour une position.

CHAPITRE 02 / M06-M10 / Q03 · Q04

Les premiers pas

Passer d’un réglage à une instruction répétée, puis commander le déplacement avec un premier if.

M06 / CODE

Règle son moteur

Consigne. Ajoute le réglage `speed`. Choisis une vitesse de 3 à 5.

Correction. Ajoute le réglage `speed` sans modifier les lignes précédentes.

```
const speed = 4;
```

À comprendre. `speed` désigne la taille d’un pas. Ce réglage seul ne déplace pas encore le héros : la mission suivante utilise sa valeur.

Réponses acceptées. Un nombre de 3 à 5, bornes incluses. 4 est une possibilité.

Test et réponse au défi. À règle de déplacement identique, un pas de 5 fait avancer plus vite qu’un pas de 3.

Erreur fréquente. Ne confonds pas `speed` (taille du pas) et `startX` (position de départ).

Q03 / BILAN QCM

Un départ ou un pas ?

Q03 / Question 1. Quelle donnée change la taille d’un pas ?

```
const startX = 40;
const speed = 5;
```

A	**speed** RÉPONSE CORRECTE	speed règle le nombre de pixels parcourus à chaque pas.
B	**startX**	startX place le départ ; speed règle le pas.
C	**playerName**	playerName contient le nom du héros, pas sa vitesse.

Q03 / Question 2. Quelle modification ne change que le départ ?

A	**Changer speed**	Changer speed change la taille des pas, pas le départ.
B	**Changer startX** RÉPONSE CORRECTE	Le départ suit startX ; la taille des pas peut rester la même.
C	**Changer toutes les lignes**	Changer une seule donnée suffit et permet de comprendre son effet.

M07 / CODE

Fais son premier pas

Consigne. Dans le bloc vert, complète le calcul de `player.x`. Garde sa position actuelle et ajoute un pas de `speed`.

Correction. Complète le cadre `updateX` déjà présent. Il ne doit y avoir qu'une seule fonction de ce nom.

```
function updateX(player, keys, speed) {
  player.x = player.x + speed;
}
```

À comprendre. Le calcul lit l'ancienne position, ajoute le pas, puis range le résultat dans `player.x`. Le jeu rappelle cette fonction à chaque étape de déplacement.

Réponses acceptées. Le résultat doit dépendre de la position ET du paramètre `speed` : avec X = 40 et un pas de 2, on obtient 42 ; avec X = 100 et un pas de 5, 105.

Test et réponse au défi. Le héros avance encore tout seul à cette étape. Avec X = 50 et un pas de 4, deux passages donnent 54 puis 58.

Erreur fréquente. `player.x = speed` remplacerait la position. Ajouter toujours 4 ne tient pas compte des autres vitesses.

M08 / DÉFI PRATIQUE

Change son allure

Consigne. Essaie `speed` à `3` et laisse le héros avancer. Passe près de `5`, vérifie et compare son allure.

Un essai possible. Tester `speed = 3`, puis `speed = 5` en gardant la déclaration `const speed` existante.

À faire dans le jeu. Après chaque vérification, laisser le héros avancer d'au moins 12 pixels pour enregistrer l'essai.

Validation attendue. Deux vitesses valides différentes d'au moins 1. Un changement sans déplacement dans le jeu ne suffit pas.

À expliquer. Le même calcul utilise la nouvelle taille de pas : 5 donne une allure plus rapide que 3.

Q04 / BILAN QCM

Prévoir le prochain pas

Q04 / Question 1. Le héros est à X = 50 et speed vaut 4. Quelle est sa prochaine position ?

```
player.x = player.x + speed;
```

A **4**
On ajoute le pas à la position actuelle : on ne remplace pas la position par le pas.

B **54 RÉPONSE CORRECTE**
50 + 4 donne 54. L'ancienne position est conservée dans le calcul.

C **46**
46 correspondrait à enlever 4. Ici, on additionne.

Q04 / Question 2. Le jeu répète ce calcul deux fois depuis X = 50. Où arrive-t-on ?

```
player.x = player.x + 4;
```

A **54**
54 est le résultat d'un seul pas. Le deuxième repart de 54.

B **8**
Deux pas ajoutent 8, mais il faut aussi garder la position initiale de 50.

C **58 RÉPONSE CORRECTE**
Premier pas : 54. Deuxième pas : 58.

M09 / CODE

Donne-lui une commande

Consigne. Dans `updateX`, entoure ton calcul avec `if (keys.right) {` et `}`. Garde aussi les accolades de la fonction.

Correction. Modifie la fonction `updateX` existante. Entoure le calcul avec ce `if` ; ne recopie pas une deuxième fonction.

```
function updateX(player, keys, speed) {
  if (keys.right) {
    player.x = player.x + speed;
  }
}
```

À comprendre. `if` exécute les instructions entre ses accolades seulement si `keys.right` est vrai. Les accolades de la fonction restent autour de l'ensemble.

Réponses acceptées. Flèche droite pressée : le héros avance du pas demandé. Sans flèche droite : sa position reste identique.

Test et réponse au défi. Appuyer fait avancer ; relâcher arrête le héros. La condition est testée à chaque passage.

Erreur fréquente. Il faut fermer le `if`, puis la fonction. Le calcul placé après l'accolade du `if` ferait avancer en permanence.

M10 / DÉFI PRATIQUE

Avance, puis stop !

Consigne. Vérifie, puis avance avec → d'au moins `50` pixels. Relâche la touche pour montrer que le héros s'arrête.

Un essai possible. Conserver le `if (keys.right)` autour du calcul de déplacement.

À faire dans le jeu. Vérifier, maintenir la flèche droite pour avancer d'au moins 50 pixels par rapport au départ de l'essai, puis relâcher.

Validation attendue. Le déplacement doit atteindre la distance demandée, puis le héros doit être observé immobile sans flèche droite pressée.

À expliquer. La condition commande aussi l'arrêt. Tester le cas où la condition est fausse est aussi utile que le cas vrai.

CHAPITRE 03 / M11-M15 / Q05 · Q06

Les commandes

Réutiliser une condition pour aller à gauche, puis comparer et tester la puissance du saut.

Q05 / BILAN QCM

Qui donne la commande ?

Q05 / Question 1. La flèche droite est relâchée. Le calcul dans ce if se fait-il ?

```
if (keys.right) {
  player.x = player.x + speed;
}
```

A **Oui, à chaque image**
Le if n'exécute son bloc que si keys.right est vrai.

B **Non RÉPONSE CORRECTE**
Sans la flèche droite, la condition est fausse et ce bloc est ignoré.

C **Seulement au début**
Le if teste la touche à chaque passage ; ce n'est pas une règle réservée au début.

Q05 / Question 2. Que placent les accolades du if ?

A **Le nom du héros**
Le nom du héros se place dans la donnée playerName.

B **La valeur de speed**
speed est un réglage. Les accolades délimitent les actions de la condition.

C **Les actions à faire si la condition est vraie RÉPONSE CORRECTE**
Les instructions entre { et } appartiennent à ce if.

M11 / CODE

Fais demi-tour

Consigne. Dans la même fonction, ajoute un deuxième `if` pour `keys.left`. Cette fois, enlève `speed` à `player.x`.

Correction. Dans la même fonction, conserve le `if` de droite et ajoute celui de gauche.

```
function updateX(player, keys, speed) {
  if (keys.right) {
    player.x = player.x + speed;
  }
  if (keys.left) {
    player.x = player.x - speed;
  }
}
```

À comprendre. Ajouter augmente X et va à droite. Soustraire diminue X et va à gauche. Les deux conditions sont indépendantes.

Réponses acceptées. X = 40 et pas de 2 vers la gauche donnent 38. À droite, le déplacement reste positif. Les deux flèches ensemble doivent se compenser.

Test et réponse au défi. Avec le même pas, un déplacement à droite puis à gauche revient à la position précédente, en l'absence d'obstacle.

Erreur fréquente. Un `else if` qui donne priorité à une direction quand les deux touches sont pressées ne respecte pas la règle demandée.

M12 / DÉFI PRATIQUE

Aller... et retour !

Consigne. Vérifie, puis dépasse la position `X = 120` avec →. Reviens près de ton départ avec ←.

Un essai possible. Conserver les deux conditions et leurs calculs : addition à droite, soustraction à gauche.

À faire dans le jeu. Après Vérifier, atteindre X = 120 ou davantage, puis revenir vers la valeur de `startX`.

Validation attendue. L'aller doit être observé avant le retour. Une marge de 5 pixels autour du départ est acceptée avec les vitesses autorisées.

À expliquer. Les deux sens agissent sur la même position. Le retour annule le trajet horizontal parcouru.

Q06 / BILAN QCM

Trouver le bon sens

Q06 / Question 1. X vaut 80 et speed vaut 4. Où mène ce calcul ?

```
player.x = player.x - speed;
```

A	**76, vers la gauche RÉPONSE CORRECTE** 80 - 4 donne 76 : X diminue, le héros va à gauche.
B	**84, vers la droite** 84 viendrait d'une addition. Ici, on soustrait.
C	**4, au début du niveau** Le pas est retiré à la position actuelle ; la position ne devient pas 4.

Q06 / Question 2. Le héros va à droite, puis à gauche avec le même pas. Quel est le résultat ?

A	**Il avance deux fois à droite** Le deuxième calcul enlève le pas au lieu de l'ajouter.
B	**Il revient à sa position précédente RÉPONSE CORRECTE** Ajouter un pas puis retirer le même pas annule le déplacement.
C	**Il monte** Ces deux calculs modifient X : ils changent l'horizontale, pas la hauteur.

M13 / CODE

Vise la plateforme

Consigne. Ajoute `jumpPower`, avec un nombre de `9` à `11`. Vérifie, puis essaie un saut avec `Espace`.

Correction. Ajoute le réglage `jumpPower` après le code déjà construit.

```
const jumpPower = 10;
```

À comprendre. La puissance règle l'impulsion initiale du saut. À gravité identique, une valeur plus grande fait monter plus haut.

Réponses acceptées. Un nombre de 9 à 11, bornes incluses.

Test et réponse au défi. Vérifier puis appuyer sur Espace fait sauter. Comparer 9 et 11 permet de choisir sa sensation de jeu, puis d'atteindre la première plateforme.

Erreur fréquente. Ne change pas en même temps plusieurs réglages pour comparer la puissance seule.

M14 / DÉFI PRATIQUE

Trouve ton saut

Consigne. Essaie une puissance près de 9, vérifie et saute. Essaie près de 11, vérifie et saute à nouveau.

Un essai possible. Tester `jumpPower = 9`, puis `jumpPower = 11`, sans changer la gravité.

À faire dans le jeu. Vérifier et faire un vrai saut avec Espace pour chacun des deux réglages.

Validation attendue. Deux puissances valides séparées d'au moins 1, chacune essayée par un saut.

À expliquer. Une impulsion plus forte permet de monter plus haut. Comparer un seul réglage permet d'attribuer l'effet à cette ligne.

M15 / DÉFI PRATIQUE

Pose-toi sur la plateforme

Consigne. Vérifie, puis avance vers la première plateforme. Saute et pose-toi dessus.

Un essai possible. Un réglage possible est `const jumpPower = 10;`. Garder les commandes droite et gauche.

À faire dans le jeu. Vérifier, avancer, sauter avant la première plateforme, puis ajuster la direction pour atterrir dessus.

Validation attendue. Un saut réel suivi d'un contact au sol sur la première plateforme. La survoler ne valide pas l'atterrissage.

À expliquer. La puissance et les commandes permettent d'atteindre une destination ; elles ne servent pas seulement à modifier des nombres.

CHAPITRE 04 / M16-M20 / Q07

Les sauts

Régler la gravité, comprendre return et distinguer une permission vraie d'une permission fausse.

M16 / CODE

Teste la gravité

Consigne. Ajoute `gravity`, entre `0.35` et `0.6`. Teste deux valeurs et garde ton réglage préféré.

Correction. Ajoute `gravity`. Conserve la même puissance pendant la comparaison.

```
const gravity = 0.45;
```

À comprendre. La gravité fait évoluer la vitesse verticale vers le bas. Une valeur plus petite allonge le saut et rend la chute moins rapide.

Réponses acceptées. Un nombre de 0.35 à 0.6, bornes incluses, écrit avec un point décimal.

Test et réponse au défi. À puissance identique, le saut avec 0.35 dure plus longtemps qu'avec 0.6.

Erreur fréquente. JavaScript attend `0.45`, avec un point. `0,45` n'est pas l'écriture d'un nombre décimal.

M17 / DÉFI PRATIQUE

Un saut sur la Lune

Consigne. Essaie une gravité près de `0.35`, vérifie et saute. Essaie près de `0.6`, vérifie et saute à nouveau.

Un essai possible. À puissance identique, tester `gravity = 0.35`, puis `gravity = 0.6`.

À faire dans le jeu. Vérifier et sauter pour chaque valeur. Observer le temps passé en l'air et la chute.

Validation attendue. Deux gravités valides séparées d'au moins 0.1, chacune essayée par un saut.

À expliquer. La gravité de 0.35 donne un saut plus léger et plus long. Garder la puissance fixe fait partie de la bonne méthode de comparaison.

M18 / DÉFI PRATIQUE

Atteins le sommet

Consigne. Vérifie tes réglages et rejoins la première plateforme. Saute ensuite sur la plateforme la plus haute.

Un essai possible. Une combinaison possible : `jumpPower = 10` et `gravity = 0.45`.

À faire dans le jeu. Vérifier, atteindre la première plateforme, puis sauter en avançant vers la plateforme centrale la plus haute.

Validation attendue. Un saut réel suivi d'un atterrissage sur le sommet central. Passer au-dessus ou toucher le côté ne suffit pas.

À expliquer. Plusieurs combinaisons peuvent réussir : le réglage doit être justifié par l'essai et le résultat observé.

Q07 / BILAN QCM

Prévoir un saut

Q07 / Question 1. On garde la même gravité. Quel réglage fait monter plus haut ?

A **jumpPower = 11 plutôt que 9 RÉPONSE CORRECTE**
Une puissance plus grande donne une impulsion plus forte.

B **jumpPower = 9 plutôt que 11**
Une puissance plus petite donne une impulsion moins forte.

C **Changer playerName**
Le nom ne change pas la force du saut.

Q07 / Question 2. On garde la même puissance. Avec quelle gravité retombe-t-on moins vite ?

A **0.6**
Une gravité plus grande attire plus vite vers le sol.

B **0.35 RÉPONSE CORRECTE**
Une gravité plus petite ralentit la chute.

C **Les deux donnent toujours le même saut**
Changer gravity change le saut même quand jumpPower reste identique.

M19 / CODE

Envoie-le vers le haut

Consigne. Dans le nouveau bloc vert, change la réponse après `return`. Renvoie la valeur opposée de `jumpPower`.

Correction. Dans le cadre `jumpSpeed`, remplace la réponse positive par son opposé.

```
function jumpSpeed(jumpPower) {
  return -jumpPower;
}
```

À comprendre. `return` fournit la réponse de la fonction. Sur cet écran, Y augmente vers le bas : une vitesse verticale négative fait monter.

Réponses acceptées. La réponse doit dépendre du paramètre : 8 donne -8, 11 donne -11. Une réponse fixe comme -10 ne suffit pas.

Test et réponse au défi. Avec une puissance de 10, on renvoie -10. Le signe moins change le sens, sans changer la force choisie.

Erreur fréquente. Garde `return` et utilise le paramètre reçu, sans guillemets.

M20 / CODE

Empêche le double saut

Consigne. Dans `canJump`, remplace `true` par l'information `player.onGround`. Teste `Espace` au sol, puis en plein saut.

Correction. Dans `canJump`, remplace la réponse toujours vraie par l'état du héros.

```
function canJump(player) {
  return player.onGround;
}
```

À comprendre. `player.onGround` est un booléen : vrai au sol, faux dans les airs. Le renvoyer directement donne la permission de sauter.

Réponses acceptées. Avec `onGround: true`, le résultat est `true`. Avec `onGround: false`, il est `false`.

Test et réponse au défi. Un premier saut est autorisé au sol ; une nouvelle pression sur Espace pendant ce saut ne doit pas relancer le héros vers le haut.

Erreur fréquente. `return true` permettrait de sauter partout. `"false"` est du texte, pas le booléen `false`.

CHAPITRE 05 / M21-M25 / Q08 · Q09

Le score

Tester le saut interdit, puis créer un compteur qui conserve les points déjà gagnés.

M21 / DÉFI PRATIQUE

Essaie de tricher avec le saut

Consigne. Vérifie et fais un saut avec `Espace`. Pendant le saut, appuie encore sur `Espace`.

Un essai possible. La fonction correcte garde `return player.onGround;`.

À faire dans le jeu. Vérifier, sauter avec Espace, relâcher puis appuyer une deuxième fois avant de retoucher le sol.

Validation attendue. Une nouvelle tentative en l'air doit être observée alors qu'un seul saut a été effectué depuis le début de l'essai. Utiliser Rejouer si nécessaire.

À expliquer. Le deuxième appui est refusé. Une bonne vérification essaie aussi une action que la règle doit empêcher.

Q08 / BILAN QCM

Au sol ou dans les airs ?

Q08 / Question 1. En plein saut, que vaut player.onGround ?

A **true**
true signifie que le héros touche le sol. En l'air, cette information est fausse.

B **false RÉPONSE CORRECTE**
false indique que le héros ne touche pas le sol.

C **"false"**
Avec les guillemets, ce serait du texte. L'information attendue est le booléen false.

Q08 / Question 2. Quelle réponse doit donner canJump quand le héros est en l'air ?

```
return player.onGround;
```

A **true, pour voler**
La règle demandée autorise un saut seulement au sol.

B **Le nom du héros**
La fonction doit donner une réponse oui/non, pas un nom.

C **false, pour refuser le nouveau saut RÉPONSE CORRECTE**
La fonction renvoie l'information du sol : false pendant le saut.

Q08 / Question 3. Avec jumpPower = 10, que renvoie ce calcul ?

```
return -jumpPower;
```

A	**-10 RÉPONSE CORRECTE** Le signe moins donne la valeur opposée. Une vitesse négative fait monter sur l'écran.
B	**10** Le signe moins change le signe de la valeur reçue.
C	**0** Le calcul ne soustrait pas deux valeurs identiques : il donne l'opposé de jumpPower.

M22 / CODE

Fais apparaître les fruits

Consigne. Ajoute `fruitPoints` avec `1`, `2` ou `3`. Vérifie, puis ramasse un fruit.

Correction. Ajoute la valeur `fruitPoints` sans toucher au déplacement.

```
const fruitPoints = 1;
```

À comprendre. Cette donnée fixe la valeur d'un fruit. Elle est réutilisée par le compteur à chaque récolte.

Réponses acceptées. Un entier parmi 1, 2 ou 3. Chaque élève peut choisir sa valeur.

Test et réponse au défi. Si chaque fruit vaut 2 points, trois fruits rapportent 6 points, avant la prime introduite plus tard.

Erreur fréquente. Le nombre doit être entier et sans guillemets.

M23 / CODE

Répare le compteur

Consigne. Le nouveau bloc oublie les anciens points : répare son `return`. Fais un calcul avec `score` et `points`.

Correction. Répare le `return` du cadre `addPoints`.

```
function addPoints(score, points) {
  return score + points;
}
```

À comprendre. `score` est le total déjà gagné et `points` la valeur du nouveau fruit. L'addition garde les deux.

Réponses acceptées. 0 + 1 donne 1 ; 4 + 2 donne 6 ; 7 + 3 donne 10. Le résultat doit fonctionner pour plusieurs valeurs.

Test et réponse au défi. Avec un point par fruit : 1 puis 2. Avec deux points par fruit : 2 puis 4. Le deuxième fruit ajoute sa valeur au total.

Erreur fréquente. `return points` oublierait l'ancien score ; `return score` oublierait le nouveau fruit.

M24 / DÉFI PRATIQUE

Ramasse un duo

Consigne. Vérifie, puis ramasse le fruit près du départ. Ramasse un deuxième fruit et observe le nouveau score.

Un essai possible. Conserver `return score + points;` dans `addPoints`.

À faire dans le jeu. Vérifier, prendre le fruit près du départ puis un deuxième fruit sur le parcours.

Validation attendue. Au moins deux fruits récoltés et un score qui garde leur valeur cumulée. Exemples : 2 points à 1 par fruit, 4 à 2 par fruit.

À expliquer. Le compteur conserve les anciens points. Le deuxième fruit ne remplace pas le total par sa seule valeur.

M25 / DÉFI PRATIQUE

Rends les fruits précieux

Consigne. Choisis une valeur de `fruitPoints`, vérifie et ramasse un fruit. Change la valeur, vérifie et ramasse encore un fruit.

Un essai possible. Tester par exemple `fruitPoints = 1`, puis `fruitPoints = 3`.

À faire dans le jeu. Vérifier et ramasser au moins un fruit avec chacune des deux valeurs.

Validation attendue. Deux valeurs entières valides différentes. Chaque essai doit inclure une récolte réelle.

À expliquer. Le même calcul d'addition fonctionne avec plusieurs valeurs de fruit : on change la donnée, sans réécrire la règle.

Q09 / BILAN QCM

Garder les points gagnés

Q09 / Question 1. Tu as 7 points et le fruit en ajoute 2. Quelle réponse faut-il garder ?

```
return score + points;
```

A **2**
2 oublie les 7 points déjà gagnés.

B **7**
7 oublie le nouveau fruit.

C **9 RÉPONSE CORRECTE**
7 + 2 donne 9 : on garde les anciens points et les nouveaux.

Q09 / Question 2. Pourquoi return points est-il un bug pour le compteur ?

A **Il remplace le score par la valeur du dernier fruit RÉPONSE CORRECTE**
Les points déjà gagnés disparaissent au lieu d'être additionnés.

B **Il ajoute trop de points**
Il n'additionne rien : il renvoie seulement les points du fruit.

C **Il change la position du héros**
La position X n'intervient pas dans cette réponse du compteur.

CHAPITRE 06 / M26-M30 / Q10

Le trophée

Prévoir un score, accepter l'égalité pour gagner et relier le danger à l'état du feu.

M26 / DÉFI PRATIQUE

Trois fruits, six points

Consigne. Passe `fruitPoints` à `2` et vérifie. Ramasse au moins trois fruits pour atteindre `6` points.

Un essai possible. Modifier la ligne existante en `const fruitPoints = 2;`.

À faire dans le jeu. Vérifier, puis récolter au moins trois fruits. Le premier est au sol ; les suivants se trouvent sur les plateformes.

Validation attendue. `fruitPoints` doit valoir exactement 2 ; au moins trois fruits doivent être ramassés ; le score doit atteindre au moins 6.

À expliquer. Trois fruits à deux points donnent six points. Prendre d'autres fruits ensuite reste accepté.

M27 / CODE

Place le trophée

Consigne. Ajoute `targetScore`, entre `3` et `5`. Ramasse assez de fruits et rejoins le trophée.

Correction. Ajoute le réglage `targetScore`. Le trophée devient accessible avec assez de points.

```
const targetScore = 3;
```

À comprendre. L'objectif est le seuil de score à atteindre. Sa valeur est indépendante de la valeur d'un fruit.

Réponses acceptées. Un entier parmi 3, 4 ou 5.

Test et réponse au défi. Avec des fruits à 2 points et un objectif de 3, deux fruits suffisent : un score supérieur au seuil est accepté. Le binôme doit pouvoir finir le niveau.

Erreur fréquente. Ne suppose pas qu'il faut avoir exactement le score cible.

M28 / CODE

Gagne au bon moment

Consigne. Dans le nouveau bloc, corrige le signe de comparaison. Accepte un `score` égal ou supérieur à `target`.

Correction. Dans `canWin`, corrige `>` en `>=`.

```
function canWin(score, target) {
  return score >= target;
}
```

À comprendre. `>=` signifie « supérieur ou égal ». La comparaison renvoie un booléen, sans modifier le score.

Réponses acceptées. Score inférieur : refus. Score égal ou supérieur : accord. La règle doit aussi fonctionner avec un autre objectif.

Test et réponse au défi. Pour un objectif de 3 : score 2 → faux ; score 3 → vrai ; score 4 → vrai.

Erreur fréquente. `>` oublie l'égalité. `=` sert à affecter une valeur et ne remplace pas une comparaison.

Q10 / BILAN QCM

Le score exact compte

Q10 / Question 1. L'objectif vaut 3. Quel score cette règle accepte-t-elle ?

```
return score >= target;
```

A **Seulement 3**
>= accepte aussi un score supérieur à l'objectif.

B **3 et tous les scores plus grands RÉPONSE CORRECTE**
Le signe >= accepte l'égalité et les valeurs supérieures.

C **2, 3 et 4**
2 est inférieur à l'objectif de 3 : il doit être refusé.

Q10 / Question 2. Quelle erreur ferait le signe > à la place de >= ?

A **Il refuserait le score exact RÉPONSE CORRECTE**
> exige un score strictement plus grand. Le score égal serait oublié.

B **Il accepterait tous les scores**
Un score plus petit resterait refusé. C'est le cas égal qui change.

C **Il doublerait le score**
Une comparaison donne vrai ou faux ; elle n'additionne pas les points.

M29 / DÉFI PRATIQUE

Teste ton trophée

Consigne. Vérifie ton code et atteins ton objectif de points. Rejoins le trophée à droite du niveau.

Un essai possible. Conserver la comparaison `return score >= target;` et un objectif de 3 à 5.

À faire dans le jeu. Vérifier, ramasser assez de points, puis rejoindre le trophée à droite du niveau.

Validation attendue. Une victoire réelle doit être observée. Un code juste ou un score suffisant sans rejoindre le trophée ne termine pas le défi.

À expliquer. La comparaison autorise la victoire dès le seuil atteint ; il faut encore jouer jusqu'à l'arrivée.

M30 / CODE

Attention au feu !

Consigne. Dans `isDangerous`, renvoie l'information `trap.active`. Observe le feu avant de traverser.

Correction. Dans `isDangerous`, renvoie directement l'état du piège.

```
function isDangerous(trap) {
  return trap.active;
}
```

À comprendre. `trap.active` vaut vrai quand le feu est allumé et faux quand il est éteint. La règle de danger suit cet état.

Réponses acceptées. Piège actif : `true`. Piège inactif : `false`.

Test et réponse au défi. Attendre que le feu soit éteint permet de traverser sans brûlure. Le défi M31 demandera de le démontrer au sol.

Erreur fréquente. Une réponse constante rendrait le feu toujours dangereux, ou jamais dangereux.

CHAPITRE 07 / M31-M35 / Q11

Le drapeau

Tester le feu, vérifier le retour au drapeau et modifier une rangée avec un seul réglage.

M31 / DÉFI PRATIQUE

Traverse au bon moment

Consigne. Vérifie et rejoins le passage avec le feu. Traverse au sol quand le feu est éteint, sans te brûler.

Un essai possible. La fonction `isDangerous` conserve `return trap.active;`.

À faire dans le jeu. Vérifier, rejoindre le passage, se poser juste à gauche du feu, attendre qu'il soit éteint, puis traverser en marchant.

Validation attendue. Passage au sol de la zone à gauche vers la droite du feu, feu éteint à la sortie, sans aucune brûlure depuis le début de l'essai. Utiliser Rejouer après une brûlure.

À expliquer. Le danger suit une information vraie ou fausse. Sauter au-dessus du feu ne démontre pas la traversée au sol demandée.

M32 / CODE

Active le drapeau

Consigne. Dans le bloc vert, compare `playerX` à `flagX`. Le drapeau doit s'activer quand le héros l'atteint ou le dépasse.

Correction. Dans `canSave`, compare la position du héros à celle du drapeau.

```
function canSave(playerX, flagX) {
  return playerX >= flagX;
}
```

À comprendre. Atteindre ou dépasser le drapeau signifie que `playerX` est supérieur ou égal à `flagX`. Les positions sont reçues en paramètres.

Réponses acceptées. Avant le drapeau : faux. À sa position exacte et après : vrai. La comparaison doit rester correcte pour une autre position de drapeau.

Test et réponse au défi. Dans le niveau actuel, le seuil est X = 192. Après activation, une brûlure ramène le héros près de X = 195.

Erreur fréquente. Écrire seulement `playerX >= 192` ne réutilise pas le paramètre `flagX` et échoue si le drapeau change de place.

M33 / DÉFI PRATIQUE

Éprouve ton drapeau

Consigne. Vérifie et dépasse le drapeau pour l'activer. Laisse le feu te toucher : observe où le héros revient.

Un essai possible. La fonction `canSave` conserve `return playerX >= flagX;`.

À faire dans le jeu. Vérifier, dépasser le drapeau pour l'activer, puis toucher volontairement le feu allumé et observer le retour.

Validation attendue. Le drapeau doit être actif AVANT une nouvelle brûlure. Le héros réapparaît près du drapeau, autour de X = 195, au lieu du départ initial.

À expliquer. Un point de sauvegarde se teste après un échec. Sa seule activation ne prouve pas que le retour fonctionne.

Q11 / BILAN QCM

Le rôle du drapeau

Q11 / Question 1. Le héros est exactement à la position du drapeau. Cette règle l'active-t-elle ?

```
return playerX >= flagX;
```

A	**Oui RÉPONSE CORRECTE**	>= inclut l'égalité : atteindre exactement le drapeau suffit.
B	**Non**	Le signe >= compte aussi le cas où les deux positions sont égales.
C	**Seulement si le héros saute**	Cette règle compare deux positions X ; elle ne teste pas le saut.

Q11 / Question 2. Après activation, à quoi sert le drapeau quand le feu te touche ?

A	**Il enlève tous les fruits**	Le drapeau ne modifie pas la rangée de fruits.
B	**Il augmente speed**	La vitesse reste le réglage speed écrit dans le fichier.
C	**Il donne un nouveau point de départ RÉPONSE CORRECTE**	Le héros revient au drapeau au lieu de recommencer tout à gauche.

M34 / CODE

Dessine un chemin de fruits

Consigne. Ajoute `fruitGap`, entre 16 et 24. Compare deux écarts sur la première plateforme.

Correction. Ajoute `fruitGap` ; le jeu utilise cette valeur pour disposer la rangée de fruits.

```
const fruitGap = 24;
```

À comprendre. Un écart unique sert à placer plusieurs fruits. Il suffit de changer ce réglage pour changer le chemin.

Réponses acceptées. Un nombre de 16 à 24, bornes incluses.

Test et réponse au défi. Avec un écart de 16, les fruits sont plus rapprochés qu'avec 24. Un écart court facilite une récolte groupée.

Erreur fréquente. Il n'est pas demandé d'écrire une boucle ou une liste : le moteur de l'atelier place la rangée.

M35 / DÉFI PRATIQUE

Dessine deux chemins

Consigne. Vérifie avec un écart près de 16. Change la même ligne vers 24 et compare la rangée.

Un essai possible. Tester `fruitGap = 16`, puis `fruitGap = 24` sur la même ligne.

À faire dans le jeu. Vérifier chaque réglage et comparer la rangée de la première plateforme.

Validation attendue. Deux écarts valides séparés d'au moins 4. Il n'est pas nécessaire de ramasser les fruits pour enregistrer ces deux dispositions.

À expliquer. 16 rapproche les fruits ; 24 les éloigne. Le jeu réutilise la même donnée pour plusieurs positions.

CHAPITRE 08 / M36-M40 / Q12

Ta version du jeu

Calculer une prime, combiner deux conditions, personnaliser le jeu et signer sa victoire.

M36 / CODE

Double la récompense

Consigne. Dans `fruitBonus`, calcule le double de `count`. Change la réponse après `return`.

Correction. Dans `fruitBonus`, calcule le double du nombre reçu.

```
function fruitBonus(count) {
  return count * 2;
}
```

À comprendre. `count` est le nombre de fruits ramassés. `*` multiplie : deux points de prime pour chaque fruit.

Réponses acceptées. 0 fruit donne 0 ; 3 fruits donnent 6 ; 5 fruits donnent 10.

Test et réponse au défi. La prime s'ajoute au score de récolte à la victoire. Avec trois fruits à 2 points, on a 6 de score et 6 de prime.

Erreur fréquente. `count + 2` ajoute seulement deux points au nombre de fruits ; une réponse fixe ne suit pas la récolte.

M37 / DÉFI PRATIQUE

Encaisse ta prime

Consigne. Vérifie et ramasse au moins trois fruits. Gagne une partie et retrouve la prime à l'arrivée.

Un essai possible. La fonction `fruitBonus` conserve `return count * 2;`.

À faire dans le jeu. Vérifier, récolter au moins trois fruits, puis gagner et lire la prime affichée à l'arrivée.

Validation attendue. Une victoire avec au moins trois fruits et une prime égale au nombre de fruits multiplié par 2.

À expliquer. Trois fruits donnent une prime de 6 ; cinq donnent 10. Cette prime est distincte du score obtenu au ramassage.

M38 / CODE

Déverrouille l'arrivée

Consigne. Le bloc vérifie déjà le score. Ajoute `&& flag` pour exiger aussi le drapeau.

Correction. Dans `canOpen`, conserve la comparaison et ajoute `&& flag`.

```
function canOpen(score, target, flag) {
  return score >= target && flag;
}
```

À comprendre. `&&` signifie « et ». Le score suffisant et le drapeau activé doivent être vrais ensemble.

Réponses acceptées. Seul le cas « score suffisant + drapeau actif » ouvre l'arrivée. Les trois autres combinaisons restent fermées.

Test et réponse au défi. Avec assez de points mais sans drapeau, la réponse est fausse. Avec le drapeau mais trop peu de points, elle est aussi fausse.

Erreur fréquente. `||` voudrait dire « ou » et accepterait une seule condition : ce n'est pas la règle du niveau.

M39 / DÉFI PRATIQUE

Crée ta version du jeu

Consigne. Vérifie et fais un saut pour garder un premier essai. Change deux réglages parmi `speed`, `jumpPower`, `gravity`, puis vérifie et saute.

Un essai possible. Essai A : `speed = 3`, `jumpPower = 9`, `gravity = 0.6`. Essai B : `speed = 4`, `jumpPower = 10`, `gravity = 0.45`.

À faire dans le jeu. Vérifier et sauter avec le premier ensemble, puis changer au moins deux des trois réglages, vérifier et sauter à nouveau.

Validation attendue. Deux ensembles valides, chacun essayé par un saut, avec au moins deux valeurs différentes parmi `speed`, `jumpPower` et `gravity`.

À expliquer. Ici, l'objectif est de créer une version personnelle en combinant les réglages connus. Pour étudier l'effet précis d'une seule ligne, on revient à un changement à la fois.

Q12 / BILAN QCM

Prêt à signer ton jeu ?

Q12 / Question 1. Tu as assez de points, mais le drapeau est inactif. L'arrivée s'ouvre-t-elle ?

```
return score >= target && flag;
```

A	**Oui, le score suffit**	&& demande que les deux conditions soient vraies.
B	**Non, il manque le drapeau RÉPONSE CORRECTE**	Le score et le drapeau sont nécessaires ensemble.
C	**Oui, si le nom est assez long**	Le nom ne fait pas partie de cette condition.

Q12 / Question 2. Tu as ramassé 5 fruits. Quelle prime renvoie ta fonction ?

```
return count * 2;
```

A **7**
7 serait le résultat de 5 + 2. Ici, le signe * multiplie.

B **5**
La réponse doit donner deux points pour chacun des cinq fruits.

C **10 RÉPONSE CORRECTE**
5 multiplié par 2 donne une prime de 10 points.

Q12 / Question 3. Pour savoir quel réglage change le saut, quelle méthode utilises-tu ?

A **Je change tout le fichier en une fois**
Avec plusieurs changements, on ne sait plus lequel a produit l'effet.

B **Je change un réglage, puis je vérifie et je joue RÉPONSE CORRECTE**
Un changement à la fois permet de relier une ligne à ce que l'on observe.

C **Je recopie sans regarder le jeu**
Le résultat dans le jeu sert à comprendre et tester les instructions.

M40 / CODE

Signe ta victoire

Consigne. Invente ton message après `return`. Assemble ton texte, `playerName` et `score`, puis termine une partie.

Correction. Personnalise le `return` de `victoryMessage`, puis termine réellement une partie.

```
function victoryMessage(playerName, score) {
  return "Bravo " + playerName + " ! Score : " + score;
}
```

À comprendre. Avec du texte, `+` assemble les morceaux. Les mots sont entre guillemets ; les paramètres apportent les informations de la partie.

Réponses acceptées. Une chaîne de texte qui contient le nom ET le score reçus, y compris si ces valeurs changent. Les mots et la ponctuation du message sont libres.

Test et réponse au défi. Exemple pour Nova et 8 : « Bravo Nova ! Score : 8 ». Dans le jeu, le score transmis au message final inclut la prime. Gagner valide la dernière mission.

Erreur fréquente. Ne tape pas un nom ou un score fixe à la place des paramètres. Le message complet doit rester du texte.

Les réponses QCM en un coup d’œil

Les lettres correspondent à l’ordre des trois propositions affichées dans l’atelier. Les explications et les mauvaises réponses commentées figurent dans les chapitres précédents.

Bilan	Titre	Réponses, dans l’ordre
Q01	Lire une ligne	1 : A · 2 : B
Q02	Comprendre la position	1 : B · 2 : C
Q03	Un départ ou un pas ?	1 : A · 2 : B
Q04	Prévoir le prochain pas	1 : B · 2 : C
Q05	Qui donne la commande ?	1 : B · 2 : C
Q06	Trouver le bon sens	1 : A · 2 : B
Q07	Prévoir un saut	1 : A · 2 : B
Q08	Au sol ou dans les airs ?	1 : B · 2 : C · 3 : A
Q09	Garder les points gagnés	1 : C · 2 : A
Q10	Le score exact compte	1 : B · 2 : A
Q11	Le rôle du drapeau	1 : A · 2 : C
Q12	Prêt à signer ton jeu ?	1 : B · 2 : C · 3 : B

Ce qu’on vérifie au-delà de la bonne lettre

L’élève sait expliquer la différence entre du texte et un nombre, prévoir le résultat d’un calcul, lire un if et reconnaître une réponse vraie ou fausse. Il peut justifier >= pour accepter l’égalité et && pour exiger deux conditions ensemble.

Une bonne réponse peut être suivie d’une reformulation orale : « Explique sans relire les choix » ou « Donne un autre exemple qui suit la même règle ».

Le fichier game.js complet

Une version finale valide. Le nom et les réglages choisis restent personnalisables dans les limites des missions. Les deux pages de code ci-dessous forment un seul fichier.

1 / 2 - Le héros, ses mouvements et son score

```
const playerName = "Nova";

const startX = 55;

const speed = 4;

function updateX(player, keys, speed) {
  if (keys.right) {
    player.x = player.x + speed;
  }
  if (keys.left) {
    player.x = player.x - speed;
  }
}

const jumpPower = 10;

const gravity = 0.45;

function jumpSpeed(jumpPower) {
  return -jumpPower;
}

function canJump(player) {
  return player.onGround;
}

const fruitPoints = 2;

function addPoints(score, points) {
  return score + points;
}

const targetScore = 3;
```

Continuer à la page suivante, dans le même fichier.

game.js - suite

2 / 2 - Les règles du niveau et la victoire

```
function canWin(score, target) {
  return score >= target;
}

function isDangerous(trap) {
  return trap.active;
}

function canSave(playerX, flagX) {
  return playerX >= flagX;
}

const fruitGap = 24;

function fruitBonus(count) {
  return count * 2;
}

function canOpen(score, target, flag) {
  return score >= target && flag;
}

function victoryMessage(playerName, score) {
  return "Bravo " + playerName + " ! Score : " + score;
}
```

Les réglages de cette version

Donnée	Valeur d'exemple	Effet
`playerName`	"Nova"	Nom affiché
`startX`	55	Départ dans la zone du défi M05
`speed`	4	Taille d'un pas
`jumpPower` / `gravity`	10 / 0.45	Puissance du saut / chute
`fruitPoints` / `targetScore`	2 / 3	Points par fruit / seuil de victoire
`fruitGap`	24	Écart de la rangée de fruits

Ce fichier contient les règles écrites par les élèves. Le dessin, les collisions et les appels répétés des fonctions sont fournis par l'atelier. Il se joue dans l'atelier : ce fichier seul n'est pas une application autonome.

Les règles à tester pour corriger

Quelques cas permettent de vérifier une compréhension réelle sans exiger une écriture identique au modèle.

Règle	Cas à essayer	Réponse attendue
Déplacement	X = 50, pas = 4, droite	X = 54
Déplacement	X = 80, pas = 4, gauche	X = 76
Commande	Aucune flèche pressée	X reste identique
Deux flèches	Droite et gauche ensemble	Les deux pas se compensent
Saut	`jumpSpeed(10)`	-10
Saut autorisé	`player.onGround` vrai / faux	true / false
Score	`addPoints(7, 2)`	9
Victoire, cible = 3	Scores 2 / 3 / 4	false / true / true
Feu	`trap.active` vrai / faux	true / false
Drapeau en X = 192	Héros en X = 191 / 192 / 193	false / true / true
Prime	0 / 3 / 5 fruits	0 / 6 / 10

La porte : les quatre combinaisons

Assez de points ?	Drapeau activé ?	Arrivée ouverte ?
Non	Non	Non
Non	Oui	Non
Oui	Non	Non
Oui	Oui	Oui

Message final. Changer les valeurs reçues : Nova et 8, puis Luna et 27. Le message doit afficher chaque fois le bon nom et le bon score, sans changer sa fonction.

Quand un élève reste bloqué

Ce qu'on observe	Ce qu'il faut regarder avec l'élève
Le nom ne fonctionne pas	Le texte a-t-il ses deux guillemets ? playerName est-il écrit avec les bonnes majuscules ?
Un réglage est refusé	La valeur est-elle un nombre sans guillemets, dans l'intervalle demandé ? Les décimaux utilisent-ils un point ?
Le jeu annonce une erreur de syntaxe	Vérifier les parenthèses, les accolades et les guillemets par paires. Relire la dernière zone modifiée.
Une déclaration existe déjà	Modifier la ligne précédente plutôt qu'ajouter un deuxième const du même nom.
Le héros avance sans touche	Le calcul est-il vraiment à l'intérieur du if ? Le if a-t-il conservé ses accolades ?
Le compteur oublie les points	Le return additionne-t-il score et points ?
Le score exact ne permet pas de gagner	La comparaison contient-elle >= plutôt que > ?
Une fonction ne donne pas de réponse	Le return est-il présent ? Renvoie-t-il la bonne information et le bon type ?
Le code est valide, mais le défi attend	L'action demandée a-t-elle réellement été faite après Vérifier : saut, fruit, retour, victoire ou deuxième essai différent ?
Un essai de comparaison ne compte pas	Les deux valeurs sont-elles assez différentes ? Pour vitesse, saut ou fruits, le déplacement, le saut ou la récolte ont-ils été observés ?
Le héros ne répond plus après une modification	Cliquer à nouveau sur Vérifier. Le jeu reprend uniquement avec un code valide.

Les mots et signes à faire expliquer

`const` : déclarer une donnée. `function` : regrouper une règle. `if` : agir si une condition est vraie. `return` : donner une réponse. `=` : affecter une valeur. `>=` : supérieur ou égal. `&&` : et. `*` : multiplier. `true` et `false` : les deux valeurs d'un booléen.

Pour conclure la correction, demander à l'élève de montrer une ligne, de prévoir l'effet d'un petit changement et de vérifier sa prévision dans son jeu.